CO1-ASSESSMENT TOOL 1

Name: A. Abhishek Reddy

Rg no:192424192

Course Code: DSA0311

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

A. Aim: To develop a Python program to extract candidate information from resumes and identify eligible candidates.

Code:

```
import re

resumes = [
    """Name: Rahul Kumar
    Email: rahul@gmail.com
    Mobile: 9876543210
    Skills: Python, Java, SQL, NLP
    Experience: 3 years""",
    """Name: Priya Sharma
    Email: priya@gmail.com
    Mobile: 9876543211
    Skills: Java, SQL
```

```
    Experience: 1 year"""
]
for resume in resumes:
    name = re.search(r"Name:\s*(.*)", resume)
    email = re.search(r"[\w.-]+@[\w.-]+\.\w+", resume)
    mobile = re.search(r"\b[6-9]\d{9}\b", resume)
    experience = re.search(r"Experience:\s*(\d+)", resume)
    skills = re.findall(
        r"Python|Java|SQL|Machine Learning|NLP",
        resume,
        re.IGNORECASE
    )
    print("\nCandidate Profile")
    print("Name:", name.group(1) if name else "Not found")
    print("Email:", email.group() if email else "Not found")
    print("Mobile:", mobile.group() if mobile else "Not found")
    print("Skills:", ", ".join(skills) if skills else "None")
    years = int(experience.group(1)) if experience else 0
    print("Experience:", years, "years")
    if years >= 2 and any(s.lower() == "python" for s in skills):
        print("Eligibility: Eligible")
    else:
        print("Eligibility: Not Eligible")
```

Output:

```
Candidate Profile
Name: Rahul Kumar
Email: rahul@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, NLP
Experience: 3 years
Eligibility: Eligible

Candidate Profile
Name: Priya Sharma
Email: priya@gmail.com
Mobile: 9876543211
Skills: Java, SQL
Experience: 1 years
Eligibility: Not Eligible
```

Result: Candidate details were extracted using regular expressions, and candidates with at least two years of experience and Python skills were identified.

Section 2: Regular Expressions – Product Search System

1. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

A. Aim: To develop a Python program that searches product names using exact, prefix, suffix, partial, and case-insensitive keyword matching.

Code:

```
import re

products = [
    "Apple Laptop",
    "Samsung Mobile",
    "Apple iPhone",
    "HP Laptop",
    "Dell Computer",
    "Samsung Tablet"
]

def search_products(keyword, mode):
    if mode == "exact":
        pattern = r"^" + re.escape(keyword) + r"$"
    elif mode == "prefix":
        pattern = r"^" + re.escape(keyword)
```

```
    elif mode == "suffix":
        pattern = re.escape(keyword) + r"$"
    else:
        pattern = re.escape(keyword)
    matches = [
        product for product in products
        if re.search(pattern, product, re.IGNORECASE)
    ]
    print("\nSearch Type:", mode)
    print("Keyword:", keyword)
    print("Matching Products:", matches)
    print("Total Matches:", len(matches))
search_products("Apple Laptop", "exact")
search_products("Samsung", "prefix")
search_products("Laptop", "suffix")
search_products("Computer", "partial")
search_products("apple", "partial")
```

Output:

```
Search Type: exact
Keyword: Apple Laptop
Matching Products: ['Apple Laptop']
Total Matches: 1

Search Type: prefix
Keyword: Samsung
Matching Products: ['Samsung Mobile', 'Samsung Tablet']
Total Matches: 2

Search Type: suffix
Keyword: Laptop
Matching Products: ['Apple Laptop', 'HP Laptop']
Total Matches: 2

Search Type: partial
Keyword: Computer
Matching Products: ['Dell Computer']
Total Matches: 1

Search Type: partial
Keyword: apple
Matching Products: ['Apple Laptop', 'Apple iPhone']
Total Matches: 2
```

Result: The product search system successfully performed different types of keyword searches and displayed the matching products and their total count.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field.

Generate a final registration status report indicating whether the registration is successful.

A. Aim: To develop a Python program that validates a student's register number, institutional email, course code, semester, and mobile number.

Code:

```
import re

register_no = input("Enter Register Number: ")

email = input("Enter Institutional Email: ")

course = input("Enter Course Code: ")

semester = input("Enter Semester: ")

mobile = input("Enter Mobile Number: ")

valid1 = re.fullmatch(r"\d{12}", register_no)

valid2 = re.fullmatch(
    r"[\w.-]+@[\w.-]+\.\w+", email
)

valid3 = re.fullmatch(
    r"[A-Z]{2,4}\d{2,4}", course
)

valid4 = re.fullmatch(
```

```
    r"(Semester )?[1-8]", semester
)
valid5 = re.fullmatch(
    r"[6-9]\d{9}", mobile
)
print("\n--- Validation Report ---")
print("Register Number:",
      "Valid" if valid1 else "Invalid")
print("Email:",
      "Valid" if valid2 else "Invalid")
print("Course Code:",
      "Valid" if valid3 else "Invalid")
print("Semester:",
      "Valid" if valid4 else "Invalid")
print("Mobile Number:",
      "Valid" if valid5 else "Invalid")
if all([valid1, valid2, valid3, valid4, valid5]):
    print("Registration Status: Successful")
else:
    print("Registration Status: Failed")
```

Output:

```
Enter Register Number: 192424192001
Enter Institutional Email:  student@saveetha.com
Enter Course Code:  DSA03
Enter Semester: 5
Enter Mobile Number: 9876543210

--- Validation Report ---
Register Number: Valid
Email: Invalid
Course Code: Invalid
Semester: Valid
Mobile Number: Valid
Registration Status: Failed
```

Result: The registration system successfully validated all five fields using regular expressions and generated the final registration status.